Introduction to neural networks

Simple neural network

Juan Irving Vásquez
jivg.org

Centro de Innovación y Desarrollo Tecnológico en Cómputo
Instituto Politécnico Nacional

September 17, 2026©



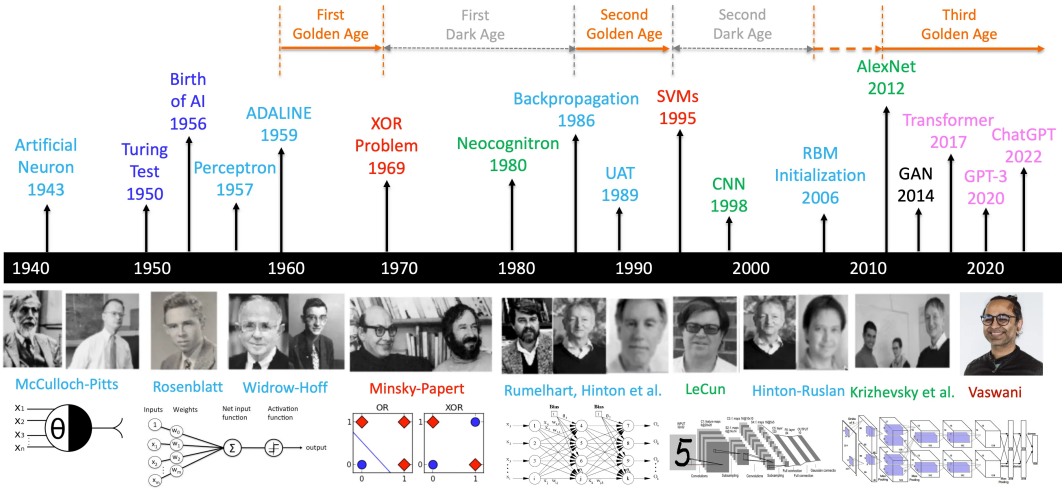
Contents

1. Modern Artificial Neuron
2. Weight learning
3. Simple Network with multiple outputs
4. Practical aspects

Roadmap

1. Modern Artificial Neuron
2. Weight learning
3. Simple Network with multiple outputs
4. Practical aspects

The big picture [1]



[1] https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b

Evolution: Rosenblatt Perceptron vs. Modern Neuron

The Rosenblatt Perceptron

- **Heaviside Step Function:** Output is strictly binary $y \in \{0, 1\}$.
- **Learning Rule:** Based on a simple heuristic error correction: $\Delta w = \eta(y - \hat{y})x$.
- **Mathematical Limitation:** The derivative is zero almost everywhere, making it non-differentiable.

The Modern Neuron

- **Continuous Activation:** Functions like Sigmoid or Tanh provide a smooth, differentiable mapping.
- **Error Surface:** Creates a continuous "loss landscape" that can be navigated.
- **Gradient Descent:** Enables optimization by iterative methods.

Modern Artificial Neuron

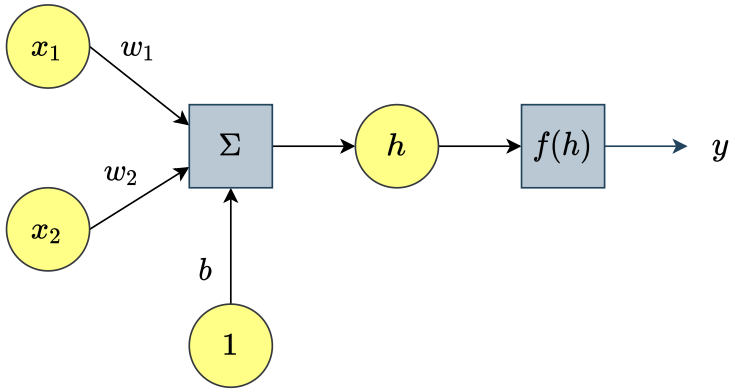
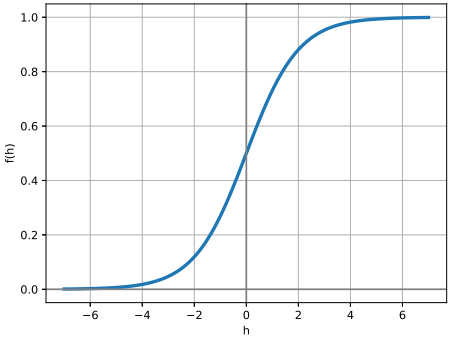


Figure: Diagram of a simple neural network (one perceptrón)

Activation functions

- The activation function can be any function.
- A nice function is the sigmoid (logistic)

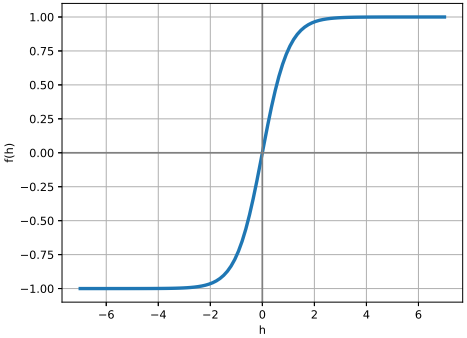
$$sigmoid(x) = \frac{1}{1 + e^{-x}} \tag{1}$$



Activation functions

- hyperbolic tangent.

$$f_{tanh}(h) = \frac{e^{h} - e^{-h}}{e^{h} + e^{-h}} \tag{2}$$



The simpest NN

$$\hat{y} = f(h) = sigmoid(\sum_{i=1}^{n} w_i x_i + b) \tag{3}$$

- A simple NN does not offer advantage over linear regression models.
- Stacking nodes provides a powerful tool versus regression models.

Evolution: Rosenblatt Perceptron vs. Modern Neuron

Key Takeaway

The shift to continuous activations moved Neural Networks from simple logic gates to powerful function approximators trainable via calculus.

Roadmap

1. Modern Artificial Neuron
2. Weight learning
3. Simple Network with multiple outputs
4. Practical aspects

Learning



Figure: Ilustración de aprendizaje. [2]

[2] Anderson-bastidas.com

Prediction Error

- How wrong the predictions are?
- Sum of squared errors (SSE)[3]

$$E = \frac{1}{2} \sum_{\mu=1}^{m} [y^{\mu} - \hat{y}^{\mu}]^2 \tag{4}$$

where $\hat{y}$ is the prediction, y is the true value and μ the data row.

[3]*divided by 2 for further derivation

Learning the weights

- Prediction

$$\hat{y}^{\mu} = f\left(\sum_{i=1}^{n} w_i x_i^{\mu} + b\right) \tag{5}$$

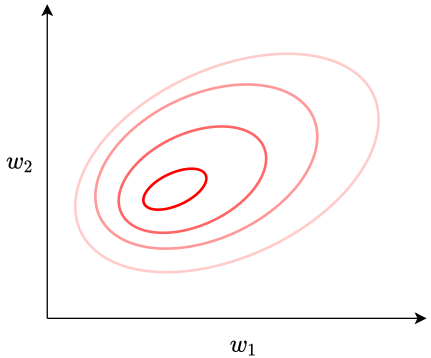
- The error depends on the weights

$$E(w_i) = \frac{1}{2}\sum_{\mu=1}^{m}\left[y^{\mu} - f\left(\sum_{i=1}^{n} w_i x_i^{\mu} + b\right)\right]^2 \tag{6}$$

- Our goal is to find the weights that minimice the error.

Gradient Descent

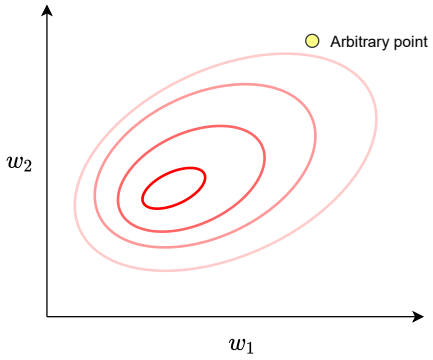
- At each step the error and the gradient are calculated, then the weights are calculated [4].



[4] Rosenbloom, P. "The method of steepest descent." Proc Symp Appl Math. Vol. 6. 1956.

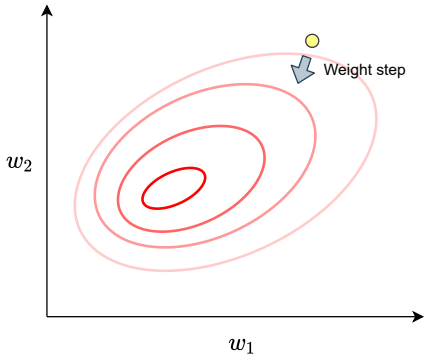
Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



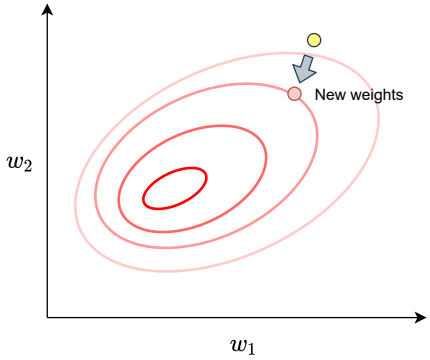
Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



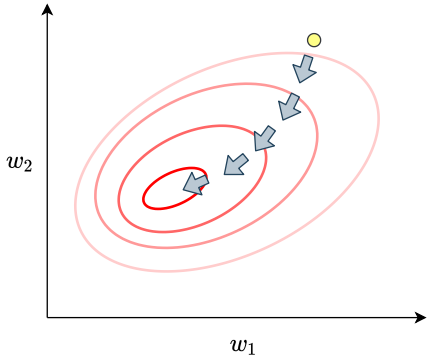
Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



Gradient Descent

- At each step the error and the gradient are calculated, then the weights are calculated.



Gradient descent

by Imcrements Method

- The gradient of the error function

$$\frac{\partial E}{\partial w_i} = -(y - \hat{y}) f'(h) x_i \tag{7}$$

- Error term

$$\delta = (y - \hat{y}) f'(h) \tag{8}$$

- Weight step (Increment)

$$\Delta w_i = \eta \delta x_i \tag{9}$$

- Weight update

$$w_i = w_i + \Delta w_i \tag{10}$$

Activation functions

- Derivative of the Sigmoid

$$f(x) = sigmoid(x) = \frac{1}{1 + e^{-x}} \tag{11}$$

$$f' = f\,(1 - f) \tag{12}$$

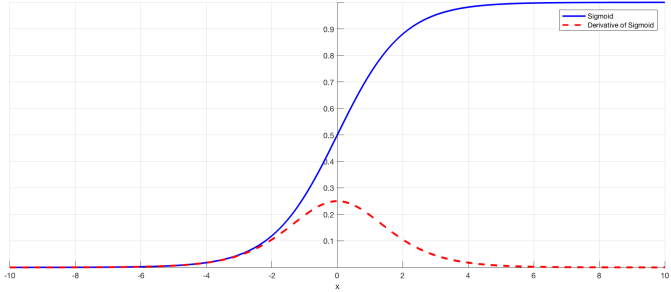


Figure: Sigmoid and its derivative

Activation functions

- Derivative of the tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{13}$$

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x) \tag{14}$$

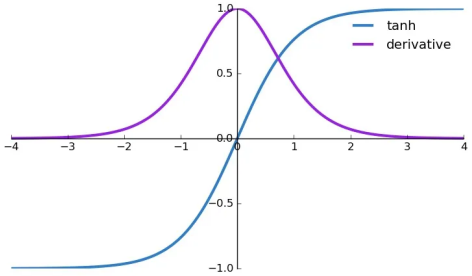


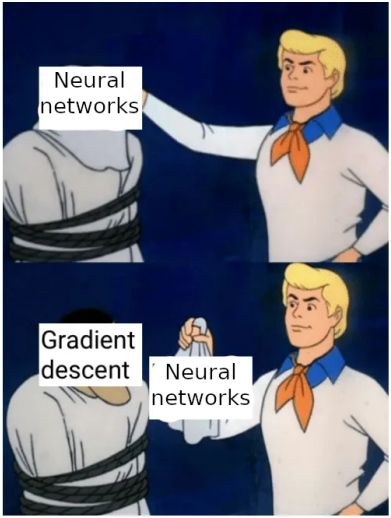
Figure: Hiperbolic tangent and its derivative

Gradient descent algorithm

Data: Datos (*Data*), número de registros (m), tasa de aprendizaje (η), épocas (*epochs*)
Result: Pesos entrenados (W)
Inicializar(W); /* Inicializar pesos */
for $e = 1$; *epochs* **do**
  Ceros(ΔW_i) ;
  foreach *(x, y)* $\in$ *Data* **do**
    $\hat{y} = f(h(x, W))$; /* Pase frontal */
    $\delta = (y - \hat{y}) f'(h)$; /* Término de error */
    foreach $w_i \in W$ **do**
      $\Delta w_i = \Delta w_i + \delta x_i$; /* Incremento */
    end
  end
  foreach $w_i \in W$ **do**
    $w_i = w_i + \frac{\eta}{m} \Delta w_i$; /* Actualización normalizada */
  end
end
return W

Algorithm 1: Descenso por gradiente

Neural networks



Roadmap

1 Modern Artificial Neuron

2 Weight learning

3 Simple Network with multiple outputs

4 Practical aspects

Multiple outputs

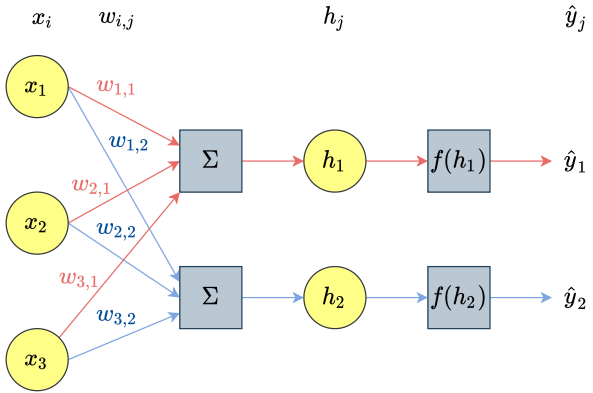


Figure: Neural network with multiple outputs

Learning the weights for multiple outputs

- Prediction

$$\hat{y}_j^\mu = f\left(\sum_i w_{i,j} x_i + b_j\right) \tag{15}$$

- Sum of squared errors (SSE)

$$E = \frac{1}{2}\sum_\mu \sum_j \left[y_j^\mu - \hat{y}_j^\mu\right]^2 \tag{16}$$

where $\hat{y}$ is the prediction, y is the true value, j the output and μ the data row. Substituting:

$$E = \frac{1}{2}\sum_\mu \sum_j \left[y_j^\mu - f\left(\sum_i w_{i,j} x_i + b\right)\right]^2 \tag{17}$$

Learning the weights for multiple outputs

- Error term

$$\delta_j = (y_j - \hat{y}_j) f'(h_j) \tag{18}$$

- Weight step

$$\Delta w_{i,j} = \eta \delta_j x_i \tag{19}$$

- Weight update

$$w_{i,j} = w_{i,j} + \Delta w_{i,j} \tag{20}$$

Roadmap

1. Modern Artificial Neuron
2. Weight learning
3. Simple Network with multiple outputs
4. Practical aspects

- Avoids the divergence from the gradient descent provoked by a large data set.

$$E = \frac{1}{m} \sum_{\mu} (y^{\mu} - \hat{y}^{\mu})^2 \tag{21}$$

- m is the number of examples

Mean absolute error

Loss functions

- Smaller penalization to large errors

$$E = \frac{1}{m} \sum_{\mu} |y^{\mu} - \hat{y}^{\mu}| \tag{22}$$

- m is the number of examples

Weights initialization

- Set bias (b) to zero
- Zero initialization for weights is not successful
- Random numbers is better but be careful of large numbers

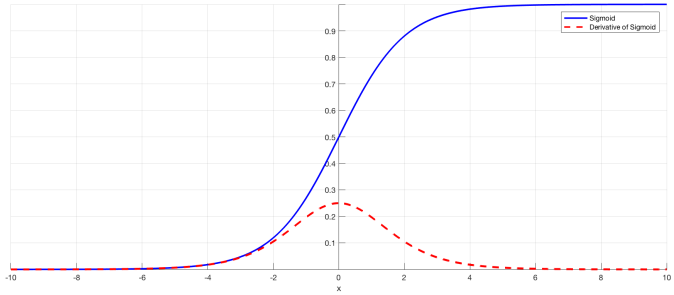


Figure: Sigmoid and its derivative

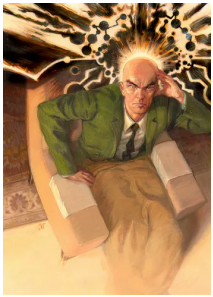
Uniform Initialization

Assigns random values to the weights within a specific range.

$$W \sim \mathcal{U}(-a, a) \tag{23}$$

where a is a value that defines the boundaries of the range.

Weights initialization (2)



- Xavier initialization [5]

$$W_{i,j} \sim U(-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}}) \tag{24}$$

where $U(-a, a)$ is the uniform distribution in the interval $(-a, a)$ and n is the number of input units.

[5] Glorot, Xavier, and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks." Proceedings of the thirteenth international conference on artificial intelligence and statistics. JMLR Workshop and Conference Proceedings, 2010.

He (Kaiming) Initialization [6]

Specially designed for layers using the **ReLU** (Rectified Linear Unit) activation function or its variants.

- Addresses the fact that ReLU is zero for half of its domain.
- For a normal distribution, the weights are sampled from:

$$W \sim \mathcal{N}\left(0, \sigma^2\right) \quad \text{where} \quad \sigma = \sqrt{\frac{2}{n_{in}}} \tag{25}$$

- For a uniform distribution, the range is:

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right) \tag{26}$$

[6] He, Kaiming, et al. "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification." Proceedings of the IEEE international conference on computer vision. 2015.

Further readings

- WIDROW, B. (1995). Perceptrons, Adalines, and Backpropagation. The Handbook of Brain Theory and Neural Networks.

References

- Vasquez Gomez, Introducción a las redes neuronales, CIDETEC-IPN
- Goodfellow, Ian, et al. Deep learning. Vol. 1. No. 2. Cambridge: MIT press, 2016.
- Zhang, Aston, et al. Dive into deep learning. Cambridge University Press, 2023.
- Stewart, James, Daniel K. Clegg, and Saleem Watson. Calculus: early transcendentals. Cengage Learning, 2020.
- Buduma, Nithin, Nikhil Buduma, and Joe Papa. Fundamentals of deep learning: Designing next-generation machine intelligence algorithms. " O'Reilly Media, Inc.", 2022.